

# Segmentation | Non-Contiguous Memory Allocation

## 1. SEPARATION OF USER VIEW AND PHYSICAL MEMORY (PAGING)

With paging, the user's way of seeing memory is different from how memory actually exists in RAM.

The user thinks in terms of variables, functions, and modules, but the OS stores data in fixed-size pages.

### Real-world example:

You write a book chapter-wise (user view), but the printer cuts the pages randomly and stores them in different boxes (physical memory).

## 2. SEGMENTATION SUPPORTS USER VIEW OF MEMORY

Segmentation is a memory management technique that matches how a programmer thinks. Programs are divided into logical parts like main function, stack, heap, libraries, etc.

### Real-world example:

A house is divided into rooms like kitchen, bedroom, and bathroom instead of equal-sized tiles.

## 3. LOGICAL ADDRESS SPACE AS COLLECTION OF SEGMENTS

In segmentation, logical memory is divided into segments based on user logic. Each segment represents a meaningful part of the program.

### Example segments:

- Main function segment
- Library functions segment
- Stack segment
- Heap segment

## 4. SEGMENT ADDRESS FORMAT <segment-number, offset> {s, d}

Each address has two parts:

- Segment number (s): which segment
- Offset (d): position inside that segment

### Real-world example:

Library → Shelf number (segment) + Book position (offset).

## 5. PROCESS DIVIDED INTO VARIABLE-SIZE SEGMENTS

Unlike paging (fixed size), segments are variable in size.

Size depends on program logic, not memory rules.

### Example:

Main function may be small, library segment may be large.

## 6. PAGING IS OS-ORIENTED, NOT USER-ORIENTED

Paging is closer to the Operating System, not the programmer.

OS divides processes into equal-size pages without caring about logic.

### Real-world example:

Cutting a movie into 10-minute clips without caring about scenes.

## 7. OS DOES NOT CARE ABOUT FUNCTION BOUNDARIES (PAGING)

A single function may be split across multiple pages.

Those pages may load at different times.

This can reduce efficiency because related code is separated.

### Real-world example:

Half of a cooking recipe is in one cupboard and the rest in another.

## 8. SEGMENTATION GROUPS SIMILAR FUNCTIONS TOGETHER

Segmentation keeps related code in the same segment.

Main function in one segment, libraries in another.

This improves performance and clarity.

### Real-world example:

Keeping all kitchen items in the kitchen, not scattered across rooms.

## 9. ADVANTAGES OF SEGMENTATION

### a. NO INTERNAL FRAGMENTATION

Memory is allocated exactly as needed.

No wasted space inside segments.

### b. CONTIGUOUS MEMORY ALLOCATION PER SEGMENT

Each segment is stored in one continuous block.

Faster execution within a segment.

### **c. SMALLER SEGMENT TABLE SIZE**

Number of segments is usually less than number of pages.  
Hence, less memory overhead than page tables.

### **d. COMPILER-FRIENDLY AND EFFICIENT**

Compiler keeps similar functions in one segment.  
Improves execution efficiency.

## **10.DISADVANTAGES OF SEGMENTATION**

### **a. EXTERNAL FRAGMENTATION**

Free memory gets broken into small holes.  
Large segment may not fit even if total memory is available.

#### **Real-world example:**

Many small empty parking spots but no space for a bus.

### **b. VARIABLE SEGMENT SIZE CAUSES SWAPPING PROBLEMS**

Different sizes make swapping slower and complex.  
Memory management becomes harder.

## **HYBRID APPROACH IN MODERN SYSTEMS**

Modern systems use both segmentation and paging together.  
Segmentation for logical structure.  
Paging inside each segment for efficient memory use.

#### **Real-world example:**

House (segments) with equal-size tiles (pages) inside rooms.